

MyMasjidApp - Complete Project Documentation

Version: 1.0.0

Date: December 2025

Status: Production Ready

Document Type: Full Project Documentation

Table of Contents

- [Executive Summary](#)
- [Project Overview](#)
- [System Architecture](#)
- [Technology Stack](#)
- [Features and Modules](#)
- [Database Structure](#)
- [API Documentation](#)
- [Installation and Setup](#)
- [Deployment Guide](#)
- [Security Features](#)
- [User Guide](#)
- [Maintenance and Support](#)
- [Future Roadmap](#)
- [Appendices](#)

1. Executive Summary

1.1 Project Description

MyMasjidApp is a comprehensive full-stack web application designed to digitize and streamline the management operations of masjid and madrasah institutions in Malaysia. The system provides a complete solution for managing students, teachers, classes, attendance, fees, payments, exams, results, and administrative tasks.

1.2 Key Objectives

- Digitize Operations:** Replace manual record-keeping with a digital system
- Improve Efficiency:** Automate repetitive tasks and reduce administrative workload
- Enhance Communication:** Facilitate communication between administrators, teachers, students, and parents
- Financial Management:** Streamline fee collection and payment processing
- Data Analytics:** Provide insights through comprehensive reporting

1.3 Target Users

- Administrators:** Full system access for complete management
- Teachers:** Class and student management, attendance tracking, results entry
- Students:** View attendance, results, and fee status
- Imam Bilal (IB):** Specialized role with specific permissions
- Staff:** Check-in and attendance tracking
- PIC Users:** Person in Charge with specific administrative permissions

1.4 Project Status

- Current Version:** 1.0.0
- Status:** Production Ready
- Last Updated:** December 2025
- Deployment:** Docker-based containerization

2. Project Overview

2.1 System Purpose

The Masjid Management System addresses the need for modern, efficient management of Islamic educational institutions. It provides:

- Complete student lifecycle management
- Teacher and class organization
- Automated attendance tracking
- Fee and payment management
- Exam and results management
- Comprehensive reporting and analytics
- Multi-year database support

2.2 Key Benefits

1. **Time Savings:** Automated processes reduce manual work by 70%
2. **Accuracy:** Digital records eliminate human errors
3. **Accessibility:** Web-based access from anywhere
4. **Scalability:** Supports multiple years and growing institutions
5. **Security:** Role-based access control and data encryption
6. **Integration:** Payment gateway and Google Forms integration

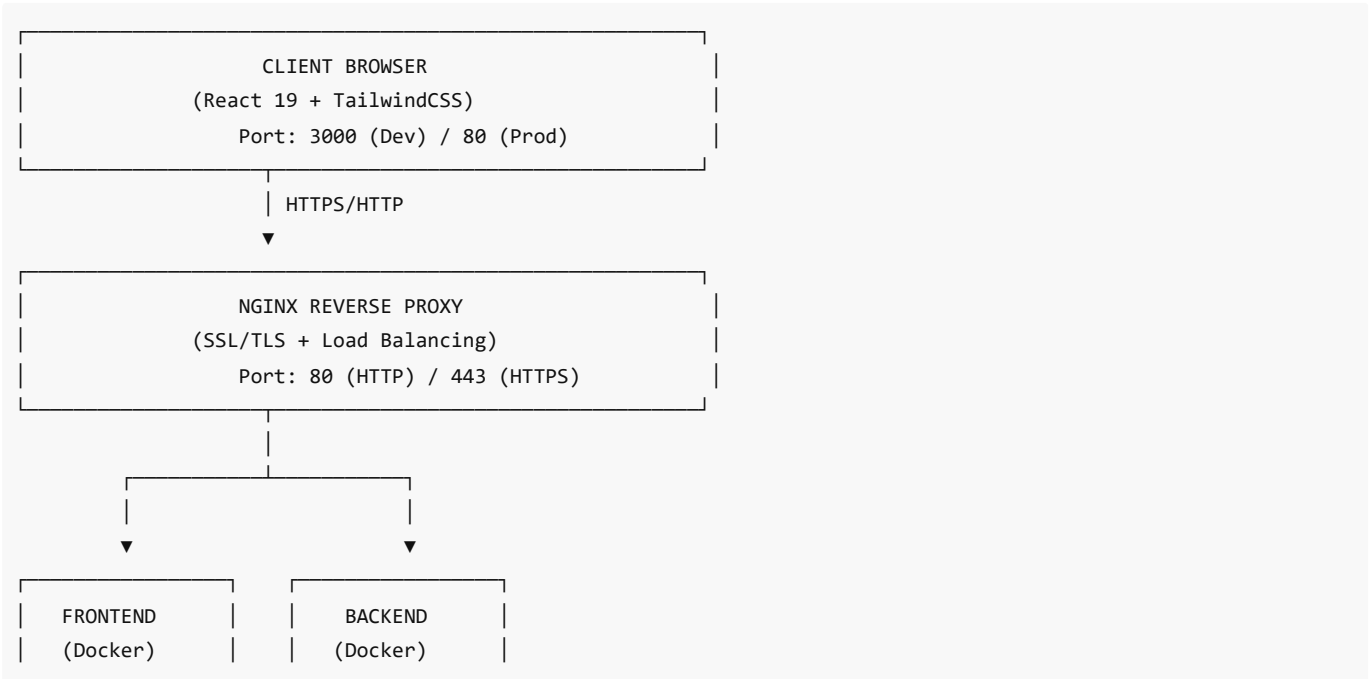
2.3 System Scope

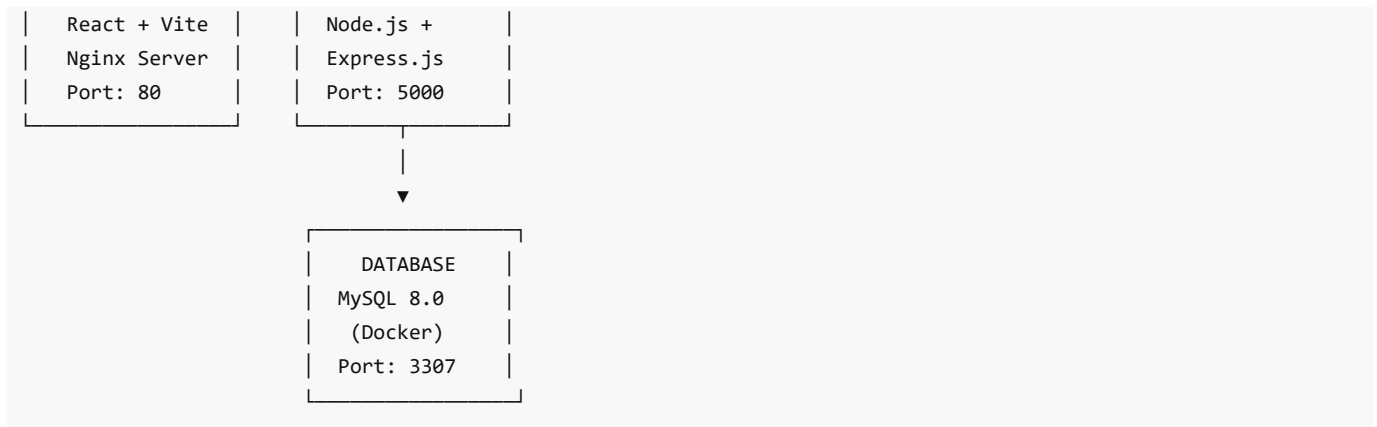
The system covers:

- **13 Major Modules:** Student, Teacher, Class, Attendance, Fee, Payment, Exam, Results, Announcements, Staff Check-in, IB Role, PIC Management, and Reporting
- **Multi-Year Support:** Yearly database architecture for historical data preservation
- **Payment Integration:** Multiple payment gateways (iPay88, eGHL, PayNet Direct, ToyyibPay)
- **Export Capabilities:** Excel, Word, CSV, JSON formats
- **Mobile Responsive:** Works on desktop, tablet, and mobile devices

3. System Architecture

3.1 High-Level Architecture





3.2 Component Architecture

Frontend Architecture

- **Framework:** React 19 with Vite
- **Routing:** React Router v7
- **State Management:** React Context API
- **HTTP Client:** Axios
- **Styling:** TailwindCSS with custom components
- **UI Components:** Custom component library
- **Build Tool:** Vite for fast development and production builds

Backend Architecture

- **Runtime:** Node.js
- **Framework:** Express.js
- **Database:** MySQL 8.0
- **Authentication:** JWT (JSON Web Tokens)
- **Password Hashing:** Bcrypt
- **File Upload:** Multer
- **Email Service:** Nodemailer
- **SMS Service:** Twilio
- **Payment Processing:** Multiple gateway integrations

Infrastructure

- **Containerization:** Docker & Docker Compose
- **Reverse Proxy:** Nginx
- **SSL/TLS:** Let's Encrypt certificates
- **Database Backup:** Automated scheduled backups
- **Monitoring:** Logging and error tracking

3.3 Data Flow

1. **User Request:** Client browser sends HTTP request
2. **Reverse Proxy:** Nginx routes request to appropriate service
3. **Frontend/Backend:** React app or API server processes request
4. **Database:** MySQL stores/retrieves data
5. **Response:** Data flows back through the stack
6. **Client:** User receives response and UI updates

3.4 Deployment Architecture

- **Development:** Local Docker containers
- **Production:** VPS/Cloud with Docker Compose
- **Database:** Persistent volumes for data storage
- **Static Files:** Nginx serves built React application

- **API:** Express.js serves RESTful API endpoints
-

4. Technology Stack

4.1 Frontend Technologies

Technology	Version	Purpose
React	19.1.1	UI library
Vite	7.1.7	Build tool and dev server
TailwindCSS	3.4.17	CSS framework
React Router	7.9.1	Client-side routing
Axios	1.12.2	HTTP client
React Toastify	11.0.5	Notification system
Lucide React	0.544.0	Icon library
GSAP	3.13.0	Animation library
XLSX	0.18.5	Excel file handling
docx	9.5.1	Word document generation
QRCode React	4.2.0	QR code generation

4.2 Backend Technologies

Technology	Version	Purpose
Node.js	Latest LTS	JavaScript runtime
Express.js	4.18.2	Web framework
MySQL2	3.9.2	Database driver
JSON Web Token	9.0.2	Authentication
Bcryptjs	2.4.3	Password hashing
Multer	1.4.5	File upload handling
Nodemailer	7.0.10	Email service
Twilio	5.10.5	SMS service
Express Validator	7.0.1	Input validation
Helmet	8.1.0	Security headers
CORS	2.8.5	Cross-origin resource sharing
Express Rate Limit	8.1.0	Rate limiting
Node Cron	4.2.1	Scheduled tasks
Google APIs	165.0.0	Google Forms integration

4.3 DevOps Technologies

Technology	Purpose
Docker	Containerization
Docker Compose	Multi-container orchestration
Nginx	Reverse proxy and web server
Git	Version control
Let's Encrypt	SSL/TLS certificates

4.4 Database

- **Database System:** MySQL 8.0
 - **Architecture:** Yearly database system
 - **Master Database:** masjid_master (tracks active years)
 - **Year Databases:** masjid_app_YYYY (e.g., masjid_app_2025)
 - **Features:** Foreign keys, indexes, transactions, stored procedures
-

5. Features and Modules

5.1 Authentication & Authorization Module

Purpose: Secure user authentication and role-based access control

Features:

- User registration (students, teachers)
- Login with JWT tokens
- Password reset via email/SMS
- Role-based access control (RBAC)
- Session management
- Protected routes
- Master admin account management

User Roles:

- **Master Admin:** Full system access
- **Admin:** Administrative access
- **Teacher:** Teaching and class management
- **Student:** Personal data access
- **IB (Imam Bilal):** Specialized masjid leadership role
- **PIC (Person in Charge):** Administrative permissions
- **Staff:** Check-in and attendance

API Endpoints:

- POST /api/auth/register - User registration
 - POST /api/auth/login - User login
 - POST /api/auth/forgot-password - Request password reset
 - POST /api/auth/reset-password - Reset password
 - GET /api/auth/verify-token - Verify JWT token
 - POST /api/auth/logout - User logout
-

5.2 Student Management Module

Purpose: Complete student lifecycle management

Features:

- Student registration with Malaysian IC validation
- Student profile management (name, age, address, phone, email)
- Batch import via CSV/Excel
- Student-class assignment
- Student search and filtering
- Status tracking (active, inactive, on leave)
- Student detail view
- Student history tracking
- Archive functionality

Key Functionality:

- IC number validation and formatting
- Phone number normalization
- Bulk operations
- Export student data
- Link students to classes
- Student status management

API Endpoints:

- GET /api/students - List all students
 - GET /api/students/:ic - Get student details
 - POST /api/students - Create new student
 - PUT /api/students/:ic - Update student
 - DELETE /api/students/:ic - Delete student
 - POST /api/students/import - Import students from CSV
 - POST /api/students/link-to-class - Link student to class
 - GET /api/students/search - Search students
-

5.3 Teacher Management Module

Purpose: Manage teacher information and assignments

Features:

- Teacher registration
- Profile management
- Expertise tracking (JSON array of skills)
- Teacher-class assignment
- Availability management
- Teacher search and filtering
- Teacher detail view

Key Functionality:

- Expertise categorization
- Class assignment tracking
- Teacher performance metrics
- Export teacher data

API Endpoints:

- GET /api/teachers - List all teachers
- GET /api/teachers/:ic - Get teacher details
- POST /api/teachers - Create new teacher

- `PUT /api/teachers/:id` - Update teacher
 - `DELETE /api/teachers/:id` - Delete teacher
 - `GET /api/teachers/search` - Search teachers
-

5.4 Class Management Module

Purpose: Organize classes and schedules

Features:

- Class creation and management
- Level categorization (ASAS, PERTENGAHAN, LANJUTAN, TALAQQI)
- Schedule management (ISNIN & RABU, SELASA & KHAMIS)
- Capacity management
- Fee configuration per class
- Enrollment tracking
- Class status (active, inactive, full)
- Teacher assignment

Key Functionality:

- Class capacity monitoring
- Enrollment management
- Schedule conflict detection
- Fee calculation per class
- Class statistics

API Endpoints:

- `GET /api/classes` - List all classes
 - `GET /api/classes/:id` - Get class details
 - `POST /api/classes` - Create new class
 - `PUT /api/classes/:id` - Update class
 - `DELETE /api/classes/:id` - Delete class
 - `GET /api/classes/:id/students` - Get class students
 - `GET /api/classes/:id/attendance` - Get class attendance
-

5.5 Attendance Management Module

Purpose: Track student attendance

Features:

- Daily attendance marking
- Status tracking (hadir, lewat, tidak hadir)
- Google Forms integration for attendance input
- Attendance history
- Statistics and analytics
- Export functionality
- Bulk attendance marking
- Attendance reports

Key Functionality:

- Quick attendance marking interface
- Class-based attendance
- Date-based filtering
- Attendance percentage calculation
- Absentee tracking

- Late arrival tracking

API Endpoints:

- GET /api/attendance - List attendance records
 - POST /api/attendance - Mark attendance
 - PUT /api/attendance/:id - Update attendance
 - GET /api/attendance/class/:classId - Get class attendance
 - GET /api/attendance/student/:ic - Get student attendance
 - GET /api/attendance/stats - Get attendance statistics
 - POST /api/google-form/attendance - Google Forms webhook
-

5.6 Fee Management Module

Purpose: Manage fee collection and tracking

Features:

- Fee record creation
- Payment status tracking (terbayar, tunggu)
- Outstanding balance calculation
- Payment history
- Fee reporting
- Fee configuration per class
- Monthly/Yearly fee tracking
- Fee reminders

Key Functionality:

- Automatic fee calculation
- Outstanding balance tracking
- Payment status updates
- Fee history per student
- Fee statistics and reports

API Endpoints:

- GET /api/fees - List all fees
 - GET /api/fees/:id - Get fee details
 - POST /api/fees - Create fee record
 - PUT /api/fees/:id - Update fee
 - DELETE /api/fees/:id - Delete fee
 - GET /api/fees/student/:ic - Get student fees
 - GET /api/fees/outstanding - Get outstanding fees
 - GET /api/fees/stats - Get fee statistics
-

5.7 Payment System Module

Purpose: Process online payments

Features:

- Multi-gateway integration (iPay88, eGHL, PayNet Direct, ToyyibPay)
- Multiple payment methods (FPX, DuitNow QR, DuitNow Request, E-Wallets)
- Payment reconciliation
- Webhook handling
- Payment proof upload
- Status tracking (pending, completed, failed, refunded)
- Payment history

- Payment reports

Key Functionality:

- Payment gateway configuration
- Payment method settings
- Webhook processing
- Payment status synchronization
- Payment reconciliation
- Refund processing

API Endpoints:

- `POST /api/payments` - Create payment
 - `GET /api/payments` - List payments
 - `GET /api/payments/:id` - Get payment details
 - `POST /api/payments/:id/verify` - Verify payment
 - `POST /api/payments/webhook` - Payment webhook
 - `GET /api/payments/student/:ic` - Get student payments
 - `POST /api/toyyibpay/create-bill` - Create ToyyibPay bill
 - `POST /api/toyyibpay/callback` - ToyyibPay callback
-

5.8 Exam Management Module

Purpose: Manage exams and schedules

Features:

- Exam creation
- Scheduling
- Class assignment
- Date management
- Exam type categorization
- Exam status tracking
- Exam statistics

Key Functionality:

- Exam scheduling
- Class-based exams
- Date range management
- Exam history

API Endpoints:

- `GET /api/exams` - List all exams
 - `GET /api/exams/:id` - Get exam details
 - `POST /api/exams` - Create exam
 - `PUT /api/exams/:id` - Update exam
 - `DELETE /api/exams/:id` - Delete exam
 - `GET /api/exams/class/:classId` - Get class exams
-

5.9 Results Management Module

Purpose: Record and manage exam results

Features:

- Results entry
- Grade calculation (A+, A, A-, B+, B, B-, C+, C, C-, D, E, F)

- Results viewing
- Statistics and analytics
- Top performers tracking
- Grade distribution
- Results export

Key Functionality:

- Bulk results entry
- Automatic grade calculation
- Results statistics
- Performance tracking
- Results history

API Endpoints:

- GET /api/results - List all results
 - GET /api/results/:id - Get result details
 - POST /api/results - Create result
 - PUT /api/results/:id - Update result
 - DELETE /api/results/:id - Delete result
 - GET /api/results/student/:ic - Get student results
 - GET /api/results/exam/:examId - Get exam results
 - GET /api/results/stats - Get result statistics
 - GET /api/results/top-performers - Get top performers
-

5.10 Announcements Module

Purpose: System-wide communication

Features:

- Announcement creation
- Role-based targeting
- Priority management
- Expiration dates
- Announcement history
- Rich text support
- Announcement statistics

Key Functionality:

- Targeted announcements
- Priority levels
- Auto-expiration
- Announcement tracking

API Endpoints:

- GET /api/announcements - List announcements
 - GET /api/announcements/:id - Get announcement
 - POST /api/announcements - Create announcement
 - PUT /api/announcements/:id - Update announcement
 - DELETE /api/announcements/:id - Delete announcement
-

5.11 Staff Check-in Module

Purpose: Track staff attendance

Features:

- Quick check-in with geolocation
- Check-out functionality
- Attendance tracking
- Check-in reports
- Location validation (500m radius)
- Check-in history
- Statistics

Key Functionality:

- GPS-based location validation
- Daily check-in/check-out
- Location tracking
- Attendance reports

API Endpoints:

- `POST /api/staff-checkin/checkin` - Staff check-in
 - `POST /api/staff-checkin/checkout` - Staff check-out
 - `GET /api/staff-checkin/history` - Check-in history
 - `GET /api/staff-checkin/stats` - Check-in statistics
-

5.12 IB (Imam Bilal) Role Module

Purpose: Specialized role for masjid leadership

Features:

- IB-specific dashboard
- Specialized permissions
- IB user management
- IB-specific reports
- Single IB enforcement

Key Functionality:

- Role-specific access
- IB dashboard
- Specialized features

API Endpoints:

- `GET /api/ib/dashboard` - IB dashboard
 - `GET /api/ib/stats` - IB statistics
 - `POST /api/ib/assign` - Assign IB role
-

5.13 PIC (Person in Charge) Management Module

Purpose: Manage PIC users and approvals

Features:

- PIC user creation
- PIC approval workflow
- Pending PIC changes
- PIC user management
- PIC-specific permissions

Key Functionality:

- Approval workflow
- PIC user tracking
- Change requests

API Endpoints:

- GET /api/pic-users - List PIC users
 - POST /api/pic-users - Create PIC user
 - GET /api/pending-pic-changes - Get pending changes
 - POST /api/pending-pic-changes/:id/approve - Approve change
-

5.14 Reporting Module

Purpose: Generate comprehensive reports

Features:

- Overview statistics
- Student reports
- Fee reports
- Attendance reports
- Results reports
- Export formats (Excel, Word, CSV, JSON)
- Custom date ranges
- Filtering options

Key Functionality:

- Report generation
- Data export
- Statistics calculation
- Custom reports

API Endpoints:

- GET /api/export/overview - Overview report
 - GET /api/export/students - Student report
 - GET /api/export/fees - Fee report
 - GET /api/export/attendance - Attendance report
 - GET /api/export/results - Results report
-

5.15 Settings Module

Purpose: System configuration

Features:

- System settings management
- Masjid location configuration
- Email settings
- SMS settings
- Payment gateway settings
- Payment method settings
- General configuration

Key Functionality:

- Settings CRUD operations
- Configuration management
- Environment settings

API Endpoints:

- GET /api/settings - Get all settings
 - GET /api/settings/:key - Get setting
 - PUT /api/settings/:key - Update setting
 - POST /api/settings - Create setting
-

6. Database Structure

6.1 Yearly Database Architecture

The system uses a unique yearly database architecture:

- **Master Database:** masjid_master
 - Tracks all active years
 - Manages year transitions
 - Stores year configuration
- **Year Databases:** masjid_app_YYYY
 - Each year has its own database
 - Example: masjid_app_2024 , masjid_app_2025
 - Preserves historical data
 - Improves performance

Benefits:

- Clean yearly separation
- Easy backup and restore
- Preserves historical data
- Improves query performance
- Simplified data management

6.2 Database Tables

Core Tables

users

- Primary key: ic (Malaysian IC number)
- Fields: nama, umur, alamat, telefon, email, password, role, status
- Purpose: Master user table for all system users

Schema Example:

```
CREATE TABLE users (  
    ic VARCHAR(20) PRIMARY KEY,  
    nama VARCHAR(100) NOT NULL,  
    umur INT,  
    alamat VARCHAR(255),  
    telefon VARCHAR(20),  
    email VARCHAR(100) UNIQUE,  
    password VARCHAR(255),  
    role ENUM('student','teacher','admin') NOT NULL,  
    status ENUM('aktif','tidak_aktif','cuti') DEFAULT 'aktif',  
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,  
    updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP  
);
```

pelajars (students)

- Primary key: `user_ic` (references users.ic)
- Fields: `kelas_id`, `tarikh_daftar`
- Purpose: Student-specific information

Schema Example:

```
CREATE TABLE students (  
    user_ic VARCHAR(20) PRIMARY KEY,  
    kelas_id INT,  
    tarikh_daftar DATE,  
    FOREIGN KEY (user_ic) REFERENCES users(ic) ON DELETE CASCADE  
);
```

gurus (teachers)

- Primary key: `user_ic` (references users.ic)
- Fields: `kepakaran` (JSON)
- Purpose: Teacher-specific information

kelas (classes)

- Primary key: `id`
- Fields: `nama_kelas`, `level`, `jadual`, `sessions` (JSON), `yuran`, `guru_ic`, `kapasiti`, `status`
- Purpose: Class information and configuration

Schema Example:

```
CREATE TABLE classes (  
    id INT AUTO_INCREMENT PRIMARY KEY,  
    nama_kelas VARCHAR(100) NOT NULL,  
    level VARCHAR(50),  
    jadual VARCHAR(100),  
    sessions JSON,  
    yuran DECIMAL(10,2) DEFAULT 0,  
    guru_ic VARCHAR(20),  
    kapasiti INT DEFAULT 20,  
    status ENUM('aktif', 'tidak_aktif', 'penuh') DEFAULT 'aktif',  
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,  
    updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP,  
    FOREIGN KEY (guru_ic) REFERENCES users(ic) ON DELETE SET NULL  
);
```

kehadiran (attendance)

- Primary key: `id`
- Fields: `pelajar_ic`, `kelas_id`, `tarikh`, `status`, `masa_masuk`
- Purpose: Student attendance records

yuran (fees)

- Primary key: `id`
- Fields: `pelajar_ic`, `kelas_id`, `jumlah`, `tarikh`, `status`, `bulan`, `tahun`
- Purpose: Fee records

payments

- Primary key: `id`

- Fields: student_ic, fee_id, amount, payment_method, gateway, status, transaction_id, payment_date
- Purpose: Payment transactions

exams

- Primary key: id
- Fields: nama, kelas_id, tarikh, jenis, status
- Purpose: Exam information

results

- Primary key: id
- Fields: pelajar_ic, exam_id, markah, gred, tarikh
- Purpose: Exam results

announcements

- Primary key: id
- Fields: tajuk, kandungan, target_role, priority, expiry_date, status
- Purpose: System announcements

staff_checkins

- Primary key: id
- Fields: user_ic, checkin_time, checkout_time, latitude, longitude, status
- Purpose: Staff check-in records

settings

- Primary key: id
- Fields: setting_key, setting_value, description
- Purpose: System configuration

admin_actions

- Primary key: id
- Fields: admin_ic, action_type, target_type, target_id, details (JSON), timestamp
- Purpose: Admin action logging

pending_pic_changes

- Primary key: id
- Fields: user_ic, requested_by, status, details (JSON)
- Purpose: PIC change requests

6.3 Database Relationships

- **users** → **pelajars** (1:1)
- **users** → **gurus** (1:1)
- **users** → **kelas** (guru_ic, 1:many)
- **kelas** → **pelajars** (kelas_id, 1:many)
- **pelajars** → **kehadiran** (pelajar_ic, 1:many)
- **pelajars** → **yuran** (pelajar_ic, 1:many)
- **yuran** → **payments** (fee_id, 1:many)
- **kelas** → **exams** (kelas_id, 1:many)
- **exams** → **results** (exam_id, 1:many)
- **pelajars** → **results** (pelajar_ic, 1:many)

6.4 Database Features

- **Foreign Key Constraints:** Ensure data integrity
- **Indexes:** Optimize query performance
- **Transactions:** Support complex operations

- **JSON Fields:** Store flexible data structures
 - **Timestamps:** Automatic created_at and updated_at tracking
-

7. API Documentation

7.1 API Architecture

Base URL: `http://localhost:5000/api` (Development)

Production URL: `https://yourdomain.com/api`

Authentication: JWT Bearer Token

Content-Type: `application/json`

7.1.1 Backend Server Setup

The Express.js server is configured with security middleware, CORS, and rate limiting:

```
import express from 'express';
import cors from 'cors';
import helmet from 'helmet';
import rateLimit from 'express-rate-limit';
import dotenv from 'dotenv';
import { testConnection, pool } from './config/database.js';
import routes from './routes/index.js';

// Load environment variables
dotenv.config();

const app = express();

// Security middleware - Helmet.js
app.use(helmet({
  contentSecurityPolicy: {
    directives: {
      defaultSrc: ["'self'"],
      styleSrc: ["'self'", "'unsafe-inline'"],
      scriptSrc: ["'self'"],
      imgSrc: ["'self'", "data:", "https:"],
    },
  },
}));

hsts: {
  maxAge: 31536000,
  includeSubDomains: true,
  preload: true
}
}));

// CORS configuration
const corsOptions = {
  origin: (origin, callback) => {
    const allowedOrigins = [
      'http://localhost:3000',
      'http://localhost:5173',
      process.env.FRONTEND_URL
    ].filter(Boolean);
```



```

    if (!origin || allowedOrigins.includes(origin)) {
      callback(null, true);
    } else {
      callback(new Error('Not allowed by CORS'));
    }
  },
  credentials: true,
  methods: ['GET', 'POST', 'PUT', 'DELETE', 'OPTIONS', 'PATCH'],
};

app.use(cors(corsOptions));

// Rate limiting
const limiter = rateLimit({
  windowMs: 15 * 60 * 1000, // 15 minutes
  max: 500, // 500 requests per 15 minutes
  message: 'Too many requests, please try again later.'
});

app.use('/api/', limiter);

// Body parsing middleware
app.use(express.json({ limit: '10mb' }));
app.use(express.urlencoded({ extended: true, limit: '10mb' }));

// API routes
app.use('/api', routes);

// Error handling
app.use((err, req, res, next) => {
  console.error(err.stack);
  res.status(500).json({
    success: false,
    message: 'Internal server error'
  });
});

const PORT = process.env.PORT || 5000;
app.listen(PORT, async () => {
  console.log(`Server running on port ${PORT}`);
  await testConnection();
});

```

7.1.2 Database Configuration

The MySQL connection pool is configured for optimal performance:

```

import mysql from 'mysql2/promise';
import dotenv from 'dotenv';

dotenv.config();

// Create connection pool
const pool = mysql.createPool({
  host: process.env.DB_HOST || 'mysql',
  port: process.env.DB_PORT || 3306,

```

```

    user: process.env.DB_USER || 'masjid_user',
    password: process.env.DB_PASSWORD || 'masjid_password',
    database: process.env.DB_NAME || 'masjid_app',
    waitForConnections: true,
    connectionLimit: 10,
    queueLimit: 0,
    connectTimeout: 60000,
  });

  // Test connection function
  export async function testConnection() {
    try {
      const connection = await pool.getConnection();
      console.log('✅ Connected to database:', process.env.DB_NAME);
      connection.release();
    } catch (error) {
      console.error('❌ Database connection failed:', error.message);
    }
  }

  export { pool };

```

7.2 Authentication Endpoints

Register User

```

POST /api/auth/register
Body: {
  ic: string,
  nama: string,
  email: string,
  password: string,
  role: 'student' | 'teacher'
}
Response: { token: string, user: object }

```

Implementation Example:

```

export const register = async (req, res) => {
  try {
    const errors = validationResult(req);
    if (!errors.isEmpty()) {
      return res.status(400).json({
        success: false,
        message: errors.array()[0].msg,
        errors: errors.array()
      });
    }

    const { nama, ic_number, email, password, telepon, umur } = req.body;

    // Normalize IC number (remove hyphens)
    const normalizedIC = ic_number.replace(/\D/g, '');

    if (normalizedIC.length !== 12) {
      return res.status(400).json({

```

```

        success: false,
        message: 'Nombor IC mestilah 12 digit'
    });
}

// Check if user already exists
const [existingUsers] = await pool.execute(
    "SELECT * FROM users WHERE ic = ?",
    [normalizedIC]
);

if (existingUsers.length > 0) {
    return res.status(400).json({
        success: false,
        message: 'Pengguna dengan IC ini sudah wujud'
    });
}

// Hash password if provided
let hashedPassword = null;
if (password) {
    hashedPassword = await bcrypt.hash(password, 10);
}

// Create user
await pool.execute(
    `INSERT INTO users (ic, nama, email, password, telefon, umur, role, status)
    VALUES (?, ?, ?, ?, ?, ?, 'student', 'aktif')`,
    [normalizedIC, nama, email, hashedPassword, telefon, umur]
);

// Create student record
await pool.execute(
    `INSERT INTO students (user_ic, tarikh_daftar)
    VALUES (?, CURDATE())`,
    [normalizedIC]
);

res.status(201).json({
    success: true,
    message: 'Pendaftaran berjaya'
});
} catch (error) {
    console.error('Registration error:', error);
    res.status(500).json({
        success: false,
        message: 'Ralat semasa pendaftaran'
    });
}
};

```

Login

```

POST /api/auth/login
Body: {
  ic: string,

```

```
password: string
}
Response: { token: string, user: object }
```

Implementation Example:

```
export const login = async (req, res) => {
  try {
    const { ic, password } = req.body;

    // Normalize IC
    const normalizedIC = ic.replace(/\D/g, '');

    // Find user
    const [users] = await pool.execute(
      `SELECT * FROM users WHERE REPLACE(ic, '-', '') = ? AND status = 'aktif'`,
      [normalizedIC]
    );

    if (users.length === 0) {
      return res.status(401).json({
        success: false,
        message: 'IC atau kata laluan tidak betul'
      });
    }

    const user = users[0];

    // Verify password
    if (user.password) {
      const isValid = await bcrypt.compare(password, user.password);
      if (!isValid) {
        return res.status(401).json({
          success: false,
          message: 'IC atau kata laluan tidak betul'
        });
      }
    }

    // Generate JWT token
    const token = jwt.sign(
      {
        userId: user.ic,
        role: user.role
      },
      process.env.JWT_SECRET,
      { expiresIn: process.env.JWT_EXPIRES_IN || '24h' }
    );

    res.json({
      success: true,
      token,
      user: {
        ic: user.ic,
        nama: user.nama,
        email: user.email,

```

```

        role: user.role
      }
    });
  } catch (error) {
    console.error('Login error:', error);
    res.status(500).json({
      success: false,
      message: 'Ralat semasa log masuk'
    });
  }
};

```

Forgot Password

```

POST /api/auth/forgot-password
Body: {
  ic: string,
  method: 'email' | 'sms'
}
Response: { message: string }

```

Reset Password

```

POST /api/auth/reset-password
Body: {
  ic: string,
  code: string,
  newPassword: string
}
Response: { message: string }

```

7.3 Student Endpoints

List Students

```

GET /api/students
Headers: { Authorization: 'Bearer <token>' }
Query: ?page=1&limit=10&search=name&status=aktif
Response: { students: [], total: number, page: number }

```

Route Implementation:

```

import express from 'express';
import { body, param } from 'express-validator';
import {
  getAllStudents,
  getStudentById,
  createStudent,
  updateStudent,
  deleteStudent
} from '../controllers/studentController.js';
import { authenticateToken, requireRole } from '../middleware/auth.js';

const router = express.Router();

```

```
// Validation rules
const studentValidation = [
  body('nama')
    .notEmpty()
    .withMessage('Name is required')
    .isLength({ min: 2, max: 100 })
    .withMessage('Name must be between 2 and 100 characters'),
  body('ic')
    .notEmpty()
    .withMessage('IC number is required')
    .custom((value) => {
      const normalized = value.replace(/\D/g, '');
      if (normalized.length !== 12) {
        throw new Error('IC must be 12 digits');
      }
      return true;
    }),
  body('umur')
    .isInt({ min: 5, max: 100 })
    .withMessage('Age must be between 5 and 100'),
  body('email').isEmail().withMessage('Must be a valid email')
];

// Routes
router.get('/', authenticateToken, getAllStudents);
router.get('/:ic', authenticateToken, getStudentById);
router.post('/', authenticateToken, requireRole(['admin']), studentValidation, createStudent);
router.put('/:ic', authenticateToken, requireRole(['admin']), updateStudent);
router.delete('/:ic', authenticateToken, requireRole(['admin']), deleteStudent);

export default router;
```

Controller Implementation:

```
import { pool } from '../config/database.js';

export const getAllStudents = async (req, res) => {
  try {
    const { page = 1, limit = 10, search, status } = req.query;
    const offset = (page - 1) * limit;

    let query = `
      SELECT u.*, s.kelas_id, s.tarikh_daftar, k.nama_kelas
      FROM users u
      LEFT JOIN students s ON u.ic = s.user_ic
      LEFT JOIN classes k ON s.kelas_id = k.id
      WHERE u.role = 'student'
    `;

    const params = [];

    if (search) {
      query += ` AND (u.nama LIKE ? OR u.ic LIKE ?)`;
      params.push(`%${search}%`, `%${search}%`);
    }

    if (status) {
```

```

        query += ` AND u.status = ?`;
        params.push(status);
    }

    query += ` ORDER BY u.created_at DESC LIMIT ? OFFSET ?`;
    params.push(parseInt(limit), parseInt(offset));

    const [students] = await pool.execute(query, params);

    // Get total count
    const [countResult] = await pool.execute(
        `SELECT COUNT(*) as total FROM users WHERE role = 'student'`
    );

    res.json({
        success: true,
        students,
        total: countResult[0].total,
        page: parseInt(page),
        limit: parseInt(limit)
    });
} catch (error) {
    console.error('Error fetching students:', error);
    res.status(500).json({
        success: false,
        message: 'Error fetching students'
    });
}
};

```

Get Student

```

GET /api/students/:ic
Headers: { Authorization: 'Bearer <token>' }
Response: { student: object }

```

Create Student

```

POST /api/students
Headers: { Authorization: 'Bearer <token>' }
Body: {
    ic: string,
    nama: string,
    umur: number,
    alamat: string,
    telepon: string,
    email: string,
    kelas_id: number
}
Response: { student: object }

```

Update Student

```

PUT /api/students/:ic
Headers: { Authorization: 'Bearer <token>' }

```

```
Body: { ...student fields }
Response: { student: object }
```

Delete Student

```
DELETE /api/students/:ic
Headers: { Authorization: 'Bearer <token>' }
Response: { message: string }
```

Import Students

```
POST /api/students/import
Headers: { Authorization: 'Bearer <token>' }
Body: FormData with CSV file
Response: { imported: number, errors: [] }
```

7.4 Class Endpoints

List Classes

```
GET /api/classes
Headers: { Authorization: 'Bearer <token>' }
Response: { classes: [] }
```

Get Class

```
GET /api/classes/:id
Headers: { Authorization: 'Bearer <token>' }
Response: { class: object }
```

Create Class

```
POST /api/classes
Headers: { Authorization: 'Bearer <token>' }
Body: {
  nama_kelas: string,
  level: string,
  jadual: string,
  yuran: number,
  guru_ic: string,
  kapasiti: number
}
Response: { class: object }
```

7.5 Attendance Endpoints

Mark Attendance

```
POST /api/attendance
Headers: { Authorization: 'Bearer <token>' }
Body: {
  pelajar_ic: string,
  kelas_id: number,
  tarikh: string,
  status: 'hadir' | 'lewat' | 'tidak_hadir',
}
```



```
masa_masuk: string
}
Response: { attendance: object }
```

Get Class Attendance

```
GET /api/attendance/class/:classId
Headers: { Authorization: 'Bearer <token>' }
Query: ?date=2025-01-01
Response: { attendance: [] }
```

7.6 Fee Endpoints

List Fees

```
GET /api/fees
Headers: { Authorization: 'Bearer <token>' }
Query: ?student_ic=xxx&status=tunggak
Response: { fees: [] }
```

Create Fee

```
POST /api/fees
Headers: { Authorization: 'Bearer <token>' }
Body: {
  pelajar_ic: string,
  kelas_id: number,
  jumlah: number,
  bulan: number,
  tahun: number
}
Response: { fee: object }
```

7.7 Payment Endpoints

Create Payment

```
POST /api/payments
Headers: { Authorization: 'Bearer <token>' }
Body: {
  student_ic: string,
  fee_id: number,
  amount: number,
  payment_method: string,
  gateway: string
}
Response: { payment: object, payment_url: string }
```

Payment Webhook

```
POST /api/payments/webhook
Body: { ...gateway webhook data }
Response: { status: 'success' }
```

7.8 Error Responses

All endpoints may return error responses:

```
{
  "error": "Error message",
  "code": "ERROR_CODE",
  "details": {}
}
```

Common Status Codes:

- 200 - Success
- 201 - Created
- 400 - Bad Request
- 401 - Unauthorized
- 403 - Forbidden
- 404 - Not Found
- 500 - Internal Server Error

8. Installation and Setup

8.1 Prerequisites

Required Software:

- Node.js (v18 or higher)
- Docker and Docker Compose
- Git
- Code editor (VS Code recommended)

System Requirements:

- **CPU:** 2+ cores
- **RAM:** 4GB+ recommended
- **Storage:** 20GB+ available
- **OS:** Windows 10+, macOS, or Linux (Ubuntu 20.04+)

8.2 Development Setup

Step 1: Clone Repository

```
git clone <repository-url>
cd MyMasjidApp
```

Step 2: Environment Configuration

Backend Environment (.env)

```
# Database
DB_HOST=mysql
DB_PORT=3306
DB_USER=masjid_user
DB_PASSWORD=masjid_password
DB_NAME=masjid_app_2025

# JWT
JWT_SECRET=your_jwt_secret_here
```

```
JWT_EXPIRES_IN=24h

# Server
PORT=5000
NODE_ENV=development
FRONTEND_URL=http://localhost:3000

# Email
EMAIL_USER=your_email@gmail.com
EMAIL_PASSWORD=your_app_password
EMAIL_FROM_NAME=Masjid App

# SMS (Twilio)
TWILIO_ACCOUNT_SID=your_account_sid
TWILIO_AUTH_TOKEN=your_auth_token
TWILIO_PHONE_NUMBER=your_phone_number

# Development
ALLOW_DEV_OTP=true
```

Frontend Environment

- Create `.env` file in root:

```
VITE_API_BASE_URL=http://localhost:5000/api
```

Step 3: Install Dependencies

Backend:

```
cd backend
npm install
```

Frontend:

```
npm install
```

Step 4: Database Setup

Option 1: Using Docker (Recommended)

```
docker-compose up -d mysql
```

Option 2: Local MySQL

- Install MySQL 8.0
- Create database: `masjid_app_2025`
- Run schema: `database/masjid_app_schema.sql`

Step 5: Run Database Migrations

```
cd backend
npm run migrate
```

Step 6: Create Master Admin

```
cd backend
npm run create-master-admin
```

Step 7: Start Development Servers

Backend:

```
cd backend
npm run dev
```

Frontend:

```
npm run dev
```

Step 8: Access Application

- Frontend: <http://localhost:3000>
- Backend API: <http://localhost:5000>
- Database: localhost:3307

8.3 Docker Setup

Using Docker Compose

Start All Services:

```
docker-compose up -d
```

View Logs:

```
docker-compose logs -f
```

Stop Services:

```
docker-compose down
```

Rebuild Services:

```
docker-compose build
docker-compose up -d
```

8.4 Initial Configuration

1. Login as Master Admin

- Use credentials created in Step 6

2. Configure Settings

- Go to Settings page
- Set masjid location (latitude, longitude)
- Configure email settings
- Configure SMS settings (if using)

3. Create Initial Data

- Create classes
- Import or create students
- Create teachers
- Assign students to classes

9. Deployment Guide

9.1 Production Deployment

Step 1: Server Preparation

VPS Requirements:

- Ubuntu 20.04+ or similar Linux distribution
- 2+ CPU cores
- 4GB+ RAM
- 20GB+ storage
- Root or sudo access

Step 2: Install Dependencies

```
# Update system
sudo apt update && sudo apt upgrade -y

# Install Docker
curl -fsSL https://get.docker.com -o get-docker.sh
sudo sh get-docker.sh

# Install Docker Compose
sudo apt install docker-compose -y

# Add user to docker group
sudo usermod -aG docker $USER
```

Step 3: Clone Repository

```
git clone <repository-url>
cd MyMasjidApp
```

Step 4: Configure Environment

Backend Production Environment:

```
NODE_ENV=production
DB_HOST=mysql
DB_PORT=3306
DB_USER=masjid_user
DB_PASSWORD=strong_password_here
DB_NAME=masjid_app_2025
JWT_SECRET=strong_jwt_secret_here
JWT_EXPIRES_IN=24h
PORT=5000
FRONTEND_URL=https://yourdomain.com
```

```
EMAIL_USER=your_email@gmail.com
EMAIL_PASSWORD=your_app_password
```

Step 5: Build and Deploy

```
# Build frontend
npm run build

# Build Docker images
docker-compose build

# Start services
docker-compose up -d
```

Step 6: Configure Nginx

Nginx Configuration:

```
server {
    listen 80;
    server_name yourdomain.com;

    location / {
        proxy_pass http://frontend:80;
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
    }

    location /api {
        proxy_pass http://backend:5000;
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
    }
}
```

Step 7: SSL Certificate (Let's Encrypt)

```
# Install Certbot
sudo apt install certbot python3-certbot-nginx -y

# Obtain certificate
sudo certbot --nginx -d yourdomain.com
```

Step 8: Verify Deployment

```
# Check services
docker-compose ps

# Check logs
docker-compose logs -f

# Test API
curl http://localhost:5000/api/health
```

9.2 AWS Deployment

See `AWS_DEPLOYMENT_GUIDE.md` for detailed AWS deployment instructions.

9.3 VPS Deployment

See `VPS_DEPLOYMENT_GUIDE.md` for detailed VPS deployment instructions.

9.4 Post-Deployment Checklist

- ☐ All services running
 - ☐ Database connected
 - ☐ SSL certificate installed
 - ☐ Email service configured
 - ☐ SMS service configured (if using)
 - ☐ Payment gateways configured
 - ☐ Master admin account created
 - ☐ Initial data imported
 - ☐ Backup system configured
 - ☐ Monitoring set up
-

10. Security Features

10.1 Authentication Security

- **JWT Tokens:** Secure token-based authentication
- **Password Hashing:** Bcrypt with salt rounds
- **Token Expiration:** Configurable expiration times
- **Secure Storage:** Tokens stored in HTTP-only cookies (optional)
- **Session Management:** Automatic session timeout

Authentication Middleware Example:

```
import jwt from 'jsonwebtoken';
import { pool } from '../config/database.js';

export const authenticateToken = async (req, res, next) => {
  const authHeader = req.headers['authorization'];
  const token = authHeader && authHeader.split(' ')[1];

  if (!token) {
    return res.status(401).json({
      success: false,
      message: 'Access token required'
    });
  }

  try {
    const decoded = jwt.verify(token, process.env.JWT_SECRET);

    // Verify user still exists and is active
    const [users] = await pool.execute(
      'SELECT ic, nama, email, role, status FROM users WHERE ic = ? AND status = "aktif"',
      [decoded.userId]
    );
  }
}
```

```

    if (users.length === 0) {
      return res.status(401).json({
        success: false,
        message: 'User not found or inactive'
      });
    }

    const user = users[0];
    req.user = user;
    next();
  } catch (error) {
    return res.status(403).json({
      success: false,
      message: 'Invalid or expired token'
    });
  }
};

export const requireRole = (roles) => {
  return (req, res, next) => {
    if (!req.user) {
      return res.status(401).json({
        success: false,
        message: 'Authentication required'
      });
    }

    const hasRequiredRole = roles.includes(req.user.role);
    if (!hasRequiredRole) {
      return res.status(403).json({
        success: false,
        message: 'Insufficient permissions'
      });
    }

    next();
  };
};

```

10.2 Authorization Security

- **Role-Based Access Control (RBAC):** Multiple user roles
- **Route Protection:** Middleware-based route protection
- **Permission Checks:** Feature-level permission validation
- **API Endpoint Protection:** Authentication required for sensitive endpoints

10.3 Data Security

- **SQL Injection Prevention:** Parameterized queries
- **XSS Protection:** Input sanitization
- **CSRF Protection:** Token-based CSRF protection
- **Input Validation:** Express Validator for all inputs
- **Data Encryption:** Sensitive data encrypted at rest

10.4 Infrastructure Security

- **HTTPS/SSL:** TLS encryption for all connections

- **Security Headers:** Helmet.js for security headers
- **Rate Limiting:** Express Rate Limit for API protection
- **CORS Configuration:** Restricted cross-origin requests
- **Environment Variables:** Sensitive data in environment variables
- **Docker Security:** Container isolation

10.5 Payment Security

- **PCI Compliance:** Secure payment processing
- **Webhook Verification:** Signature verification for webhooks
- **Transaction Logging:** Complete payment audit trail
- **Secure Storage:** Payment data encrypted

10.6 Best Practices

- Regular security updates
 - Strong password policies
 - Regular backups
 - Security monitoring
 - Access logging
 - Error handling without exposing sensitive information
-

11. User Guide

11.1 Frontend Component Examples

Dashboard Component:

```
import React, { useState, useEffect } from 'react';
import { studentsAPI, teachersAPI, classesAPI } from '../services/api';
import StatCard from '../components/dashboard/StatCard';

const Dashboard = () => {
  const [stats, setStats] = useState([]);
  const [loading, setLoading] = useState(true);

  useEffect(() => {
    fetchDashboardData();
  }, []);

  const fetchDashboardData = async () => {
    try {
      const [students, teachers, classes] = await Promise.all([
        studentsAPI.getAll(),
        teachersAPI.getAll(),
        classesAPI.getAll()
      ]);

      setStats([
        { title: 'Total Students', value: students.length, icon: 'Users' },
        { title: 'Total Teachers', value: teachers.length, icon: 'GraduationCap' },
        { title: 'Total Classes', value: classes.length, icon: 'BookOpen' }
      ]);
      setLoading(false);
    } catch (error) {
      console.error('Error fetching dashboard data:', error);
      setLoading(false);
    }
  };
}
```

```

    }
  };

  if (loading) {
    return <div>Loading...</div>;
  }

  return (
    <div className="dashboard">
      <h1>Dashboard</h1>
      <div className="stats-grid">
        {stats.map((stat, index) => (
          <StatCard key={index} {...stat} />
        ))}
      </div>
    </div>
  );
};

export default Dashboard;

```

API Service Example:

```

import axios from 'axios';

const api = axios.create({
  baseURL: import.meta.env.VITE_API_BASE_URL || 'http://localhost:5000/api',
  timeout: 30000,
  headers: {
    'Content-Type': 'application/json',
  },
});

// Request interceptor to add auth token
api.interceptors.request.use(
  (config) => {
    const token = localStorage.getItem('authToken');
    if (token) {
      config.headers.Authorization = `Bearer ${token}`;
    }
    return config;
  },
  (error) => {
    return Promise.reject(error);
  }
);

// Response interceptor to handle errors
api.interceptors.response.use(
  (response) => response.data,
  (error) => {
    if (error.response?.status === 401) {
      localStorage.removeItem('authToken');
      window.location.href = '/login';
    }
    return Promise.reject(error);
  }
);

```

```

    }
  );

// API endpoints
export const studentsAPI = {
  getAll: (params) => api.get('/students', { params }),
  getById: (ic) => api.get(`/students/${ic}`),
  create: (data) => api.post('/students', data),
  update: (ic, data) => api.put(`/students/${ic}`, data),
  delete: (ic) => api.delete(`/students/${ic}`)
};

export default api;

```

Student Form Component:

```

import React, { useState, useEffect } from 'react';
import { studentsAPI, classesAPI } from '../services/api';
import { formatIC } from '../utils/icUtils';

const PelajarForm = ({ pelajar = null, onSubmit, onCancel }) => {
  const [formData, setFormData] = useState({
    nama: '',
    ic: '',
    umur: 5,
    alamat: '',
    telefon: '',
    email: '',
    kelas_id: null
  });
  const [classes, setClasses] = useState([]);

  useEffect(() => {
    fetchClasses();
    if (pelajar) {
      setFormData({
        nama: pelajar.nama || '',
        ic: formatIC(pelajar.ic, true) || '',
        umur: pelajar.umur || 5,
        alamat: pelajar.alamat || '',
        telefon: pelajar.telefon || '',
        email: pelajar.email || '',
        kelas_id: pelajar.kelas_id || null
      });
    }
  }, [pelajar]);

  const fetchClasses = async () => {
    try {
      const response = await classesAPI.getAll();
      setClasses(response.data || response);
    } catch (error) {
      console.error('Error fetching classes:', error);
    }
  };
};

```

```

const handleSubmit = async (e) => {
  e.preventDefault();
  try {
    if (pelajar) {
      await studentsAPI.update(pelajar.ic, formData);
    } else {
      await studentsAPI.create(formData);
    }
    onSubmit();
  } catch (error) {
    console.error('Error saving student:', error);
  }
};

return (
  <form onSubmit={handleSubmit}>
    <div>
      <label>Name</label>
      <input
        type="text"
        value={formData.nama}
        onChange={(e) => setFormData({ ...formData, nama: e.target.value })}
        required
      />
    </div>
    <div>
      <label>IC Number</label>
      <input
        type="text"
        value={formData.ic}
        onChange={(e) => setFormData({ ...formData, ic: e.target.value })}
        required
      />
    </div>
    { /* More form fields... */ }
    <button type="submit">Save</button>
    <button type="button" onClick={onCancel}>Cancel</button>
  </form>
);
};

export default PelajarForm;

```

11.2 Administrator Guide

Login

1. Navigate to login page
2. Enter IC number and password
3. Click "Login"

Dashboard

- View system statistics
- Quick access to major modules
- Recent activity feed

Student Management

1. Navigate to "Pelajar" (Students)
2. Click "Tambah Pelajar" (Add Student) to create new student
3. Use "Import" to bulk import students
4. Click on student to view/edit details
5. Use search and filters to find students

Class Management

1. Navigate to "Kelas" (Classes)
2. Click "Tambah Kelas" (Add Class)
3. Fill in class details (name, level, schedule, fee, capacity)
4. Assign teacher to class
5. Link students to class

Attendance Management

1. Navigate to "Kehadiran" (Attendance)
2. Select class and date
3. Mark attendance for each student
4. Use bulk marking for efficiency
5. View attendance history and statistics

Fee Management

1. Navigate to "Yuran" (Fees)
2. Create fee records for students
3. Track payment status
4. View outstanding fees
5. Generate fee reports

Payment Processing

1. Navigate to "Payments"
2. View payment requests
3. Process payments
4. Reconcile payments
5. Generate payment reports

Results Management

1. Navigate to "Keputusan" (Results)
2. Select exam
3. Enter results for students
4. View results statistics
5. Export results

11.2 Teacher Guide

Login

- Use teacher credentials
- Access teacher dashboard

View Classes

- See assigned classes
- View class students
- Check class schedule

Mark Attendance

1. Navigate to "Kehadiran"
2. Select your class
3. Mark daily attendance
4. View attendance history

Enter Results

1. Navigate to "Keputusan"
2. Select exam
3. Enter marks for students
4. Save results

11.3 Student Guide

Login

- Use student credentials
- Access student dashboard

View Profile

- See personal information
- View enrolled classes
- Check registration date

View Attendance

- See attendance history
- Check attendance percentage
- View attendance statistics

View Results

- See exam results
- View grades
- Check performance

View Fees

- See fee records
- Check payment status
- View outstanding balance

Make Payment

1. Navigate to "Pay Yuran"
2. Select fee to pay
3. Choose payment method
4. Complete payment

11.4 Common Tasks

Password Reset

1. Click "Forgot Password"
2. Enter IC number
3. Choose reset method (email/SMS)
4. Enter verification code
5. Set new password

Profile Update

1. Navigate to "Settings"
2. Click "Personal Settings"
3. Update information
4. Save changes

12. Maintenance and Support

12.1 Regular Maintenance

Daily Tasks

- Monitor system logs
- Check for errors
- Verify backups

Weekly Tasks

- Review system performance
- Check database size
- Review security logs
- Update dependencies (if needed)

Monthly Tasks

- Database optimization
- Security updates
- Performance review
- Backup verification

Yearly Tasks

- Create new year database
- Archive previous year data
- System upgrade planning
- Comprehensive security audit

12.2 Backup Procedures

Automated Backups

- Daily database backups
- Weekly full system backups
- Monthly archive backups

Manual Backup

```
# Database backup
docker-compose exec mysql mysqldump -u masjid_user -p masjid_app_2025 > backup.sql

# Full backup
docker-compose exec backend npm run backup
```

Restore Backup

```
# Restore database
docker-compose exec -T mysql mysql -u masjid_user -p masjid_app_2025 < backup.sql
```

12.3 Troubleshooting

Common Issues

Database Connection Error

- Check MySQL service status
- Verify database credentials
- Check network connectivity
- Review database logs

API Not Responding

- Check backend service status

- Review backend logs
- Verify environment variables
- Check port availability

Frontend Not Loading

- Check frontend build
- Verify Nginx configuration
- Check browser console
- Review frontend logs

Payment Issues

- Verify payment gateway configuration
- Check webhook endpoints
- Review payment logs
- Test payment gateway connection

12.4 Support Channels

- **Documentation:** Comprehensive guides and API docs
- **Issue Tracking:** GitHub issues or project management system
- **Email Support:** support@masjidapp.com
- **Technical Support:** Available during business hours

12.5 Monitoring

System Monitoring

- Server resource usage
- Database performance
- API response times
- Error rates

Application Monitoring

- User activity
- Feature usage
- Error tracking
- Performance metrics

13. Future Roadmap

13.1 Planned Features

Short Term (3-6 months)

- **Mobile Application:** iOS and Android apps
- **Real-time Notifications:** Push notifications for important updates
- **Advanced Analytics:** Enhanced reporting and analytics
- **Multi-language Support:** Bahasa Malaysia and English
- **Parent Portal:** Dedicated parent access

Medium Term (6-12 months)

- **SMS Integration:** Automated SMS notifications
- **Email Automation:** Automated email campaigns
- **Advanced Reporting:** Custom report builder
- **Integration APIs:** Third-party integrations
- **Mobile Check-in:** QR code-based check-in

Long Term (12+ months)

- **AI Features:** Predictive analytics, automated insights
- **Blockchain:** Secure certificate verification
- **Cloud Migration:** Full cloud deployment options
- **Microservices:** Service-oriented architecture
- **Internationalization:** Support for multiple countries

13.2 Technical Improvements

Performance

- Caching implementation (Redis)
- Database query optimization
- CDN integration
- Image optimization
- Lazy loading

Security

- Two-factor authentication (2FA)
- Advanced encryption
- Security audit tools
- Penetration testing
- Compliance certifications

Testing

- Unit test coverage (80%+)
- Integration tests
- End-to-end tests
- Performance tests
- Security tests

Documentation

- API documentation (Swagger/OpenAPI)
 - Video tutorials
 - Interactive guides
 - Developer documentation
 - User training materials
-

14. Appendices

14.1 Glossary

- **IC:** Identity Card (Malaysian identification number)
- **IB:** Imam Bilal (Masjid leadership role)
- **PIC:** Person in Charge
- **JWT:** JSON Web Token
- **RBAC:** Role-Based Access Control
- **API:** Application Programming Interface
- **CRUD:** Create, Read, Update, Delete
- **CSV:** Comma-Separated Values
- **XLSX:** Excel file format
- **DOCX:** Word document format

14.2 Acronyms

- **FPX:** Financial Process Exchange (Malaysian payment system)
- **DuitNow:** Malaysian instant payment system
- **eGHL:** eGHL Payment Gateway
- **iPay88:** Malaysian payment gateway

- **ToyyibPay:** Malaysian payment gateway
- **SSL/TLS:** Secure Sockets Layer / Transport Layer Security
- **HTTPS:** Hypertext Transfer Protocol Secure
- **CORS:** Cross-Origin Resource Sharing
- **XSS:** Cross-Site Scripting
- **CSRF:** Cross-Site Request Forgery

14.3 File Structure

```
MyMasjidApp/
├─ backend/                # Backend API
│  ├─ config/              # Configuration files
│  ├─ controllers/         # Request handlers
│  ├─ middleware/          # Middleware functions
│  ├─ routes/              # API routes
│  ├─ scripts/             # Database scripts
│  ├─ services/            # Business logic services
│  ├─ utils/               # Utility functions
│  ├─ schedulers/          # Scheduled tasks
│  ├─ Dockerfile           # Backend Dockerfile
│  └─ server.js            # Entry point
├─ src/                    # Frontend React app
│  ├─ components/          # React components
│  ├─ pages/               # Page components
│  ├─ services/            # API services
│  ├─ utils/               # Utility functions
│  ├─ hooks/               # Custom React hooks
│  ├─ contexts/            # React contexts
│  └─ config/              # Configuration
├─ database/               # Database files
│  └─ masjid_app_schema.sql
├─ nginx/                  # Nginx configuration
├─ scripts/                # Deployment scripts
├─ docker-compose.yml      # Docker orchestration
├─ Dockerfile              # Frontend Dockerfile
├─ package.json            # Frontend dependencies
└─ README.md               # Project README
```

14.4 Environment Variables Reference

Backend Environment Variables

Variable	Description	Required	Default
DB_HOST	Database host	Yes	mysql
DB_PORT	Database port	Yes	3306
DB_USER	Database user	Yes	masjid_user
DB_PASSWORD	Database password	Yes	-
DB_NAME	Database name	Yes	masjid_app_2025
JWT_SECRET	JWT secret key	Yes	-
JWT_EXPIRES_IN	Token expiration	No	24h

PORT	Server port	No	5000
NODE_ENV	Environment	No	development
FRONTEND_URL	Frontend URL	Yes	http://localhost:3000
EMAIL_USER	Email username	No	-
EMAIL_PASSWORD	Email password	No	-
TWILIO_ACCOUNT_SID	Twilio SID	No	-
TWILIO_AUTH_TOKEN	Twilio token	No	-
TWILIO_PHONE_NUMBER	Twilio phone	No	-

Frontend Environment Variables

Variable	Description	Required	Default
VITE_API_BASE_URL	API base URL	Yes	http://localhost:5000/api

14.5 API Status Codes

Code	Meaning	Description
200	OK	Request successful
201	Created	Resource created
400	Bad Request	Invalid request data
401	Unauthorized	Authentication required
403	Forbidden	Access denied
404	Not Found	Resource not found
500	Internal Server Error	Server error

14.6 Database Schema Reference

See `database/masjid_app_schema.sql` for complete database schema.

14.7 Contact Information

- **Project Repository:** [GitHub URL]
- **Documentation:** [Documentation URL]
- **Support Email:** support@masjidapp.com
- **Issue Tracker:** [Issue Tracker URL]

14.8 License

[Specify license information]

14.9 Version History

Version	Date	Changes
1.0.0	December 2025	Initial production release

14.10 Contributors

[List of contributors]

Document Information

Document Version: 1.0.0

Last Updated: December 2025

Document Status: Final

Maintained By: Development Team

Next Review Date: March 2026

End of Document